

535240175_FastFoodClassificationPytorchDensenet_P11

May 11, 2026

```
[1]: import numpy as np
import pandas as pd

import os
for dirname, _, filenames in os.walk('./input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
[2]: %pip install torchinfo
import PIL
import torch
import torch.nn as nn
from torchvision import datasets, transforms
import re, os, sys, json, cv2, random
from tqdm import tqdm
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
from PIL import Image
from torchinfo import summary
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, f1_score, confusion_matrix, classification_report
import seaborn as sns
```

Requirement already satisfied: torchinfo in /home/kaming/Work/data-science/.venv/lib/python3.12/site-packages (1.8.0)

Note: you may need to restart the kernel to use updated packages.

```
[1]: !kaggle datasets download -d kmader/food41
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.0), please consider upgrading to the latest version (2.0.2)

Dataset URL: <https://www.kaggle.com/datasets/kmader/food41>

License(s): copyright-authors

food41.zip: Skipping, found more recently modified local copy (use --force to force download)

```
[4]: import zipfile
```

```
with zipfile.ZipFile('./food41.zip', 'r') as zip_ref:  
    zip_ref.extractall('./dataset')
```

```
[3]: root = './dataset/images'
```

```
def read_split_data(root, plot_image=False):  
    filepaths = []  
    labels = []  
    bad_images = []  
  
    random.seed(0)  
    assert os.path.exists(root), 'wdnmd'  
  
    classes = [cla for cla in os.listdir(root) if os.path.isdir(os.path.  
↪join(root, cla))]  
    classes.sort()  
    class_indices = {k: v for k, v in enumerate(classes)}  
  
    json_str = json.dumps({v: k for k, v in class_indices.items()}, indent=4)  
  
    with open('classes_indices.json', 'w') as json_file:  
        json_file.write(json_str)  
  
    every_class_num = []  
    supported = ['.jpg', '.png', '.jpeg', '.PNG', '.JPG', '.JPEG']  
  
    for klass in classes:  
        classpath = os.path.join(root, klass)  
        images = [  
            os.path.join(root, klass, i)  
            for i in os.listdir(classpath)  
            if os.path.splitext(i)[-1] in supported  
        ]  
  
        every_class_num.append(len(images))  
        flist = sorted(os.listdir(classpath))  
        desc = f'{klass:23s}'  
  
        for f in tqdm(flist[:200], ncols=110, desc=desc, unit='file',  
↪colour='blue'):  
            fpath = os.path.join(classpath, f)  
            f1 = f.lower()  
            index = f1.rfind('.')  
            ext = f1[index:]  
  
            if ext in supported:
```

```

        image = Image.open(fpath)

        if image.mode == 'RGB':
            try:
                img = cv2.imread(fpath)
                filepaths.append(fpath)
                labels.append(klass)
            except:
                bad_images.append(fpath)
                print('defective image file: ', fpath)
        else:
            bad_images.append(fpath)

Fseries = pd.Series(filepaths, name='filepaths')
Lseries = pd.Series(labels, name='labels')
df = pd.concat([Fseries, Lseries], axis=1)

print(f'{len(df.labels.unique())} kind of images were found in the dataset')

train_df, test_df = train_test_split(
    df,
    train_size=.8,
    shuffle=True,
    random_state=123,
    stratify=df['labels']
)

train_image_path = train_df['filepaths'].tolist()
val_image_path = test_df['filepaths'].tolist()

train_image_label = [class_indices[i] for i in train_df['labels'].tolist()]
val_image_label = [class_indices[i] for i in test_df['labels'].tolist()]

sample_df = train_df.sample(n=50, replace=False)

ht, wt, count = 0, 0, 0

for i in range(len(sample_df)):
    fpath = sample_df['filepaths'].iloc[i]
    try:
        img = cv2.imread(fpath)
        h = img.shape[0]
        w = img.shape[1]
        ht += h
        wt += w
        count += 1
    except:

```

```

        pass

    have = int(ht / count)
    wave = int(wt / count)
    aspect_ratio = have / wave

    print('{} images were found in the dataset.\n{} for training, {} for_
    ↪validation'.format(
        sum(every_class_num),
        len(train_image_path),
        len(val_image_path)
    ))

    print(
        'average image height= ', have,
        ' average image width= ', wave,
        ' aspect ratio h/w= ', aspect_ratio
    )

    if plot_image:
        plt.figure(figsize=(12, 5))
        plt.bar(range(20), every_class_num[:20], align='center')
        plt.xticks(range(20), classes[:20], rotation=45)
        plt.show()

        for i, v in enumerate(every_class_num):
            plt.text(x=i, y=v + 5, s=str(v), ha='center')

        plt.bar(range(len(classes)), every_class_num)
        plt.title("Class Distribution (No Labels)")
        plt.show()

    return train_image_path, train_image_label, val_image_path,
    ↪val_image_label, class_indices

train_image_path, train_image_label, val_image_path, val_image_label,
    ↪class_indices = read_split_data(
    root, plot_image=True
)

```

```

apple_pie          :
100%|               | 200/200 [00:00<00:00,
932.56file/s]
baby_back_ribs     :
100%|               | 200/200 [00:00<00:00,
909.27file/s]

```

baklava	:	
100%		200/200 [00:00<00:00,
1024.38file/s]		
beef_carpaccio	:	
100%		200/200 [00:00<00:00,
877.33file/s]		
beef_tartare	:	
100%		200/200 [00:00<00:00,
867.89file/s]		
beet_salad	:	
100%		200/200 [00:00<00:00,
655.27file/s]		
beignets	:	
100%		200/200 [00:00<00:00,
650.03file/s]		
bibimbap	:	
100%		200/200 [00:00<00:00,
615.04file/s]		
bread_pudding	:	
100%		200/200 [00:00<00:00,
775.99file/s]		
breakfast_burrito	:	
100%		200/200 [00:00<00:00,
957.32file/s]		
bruschetta	:	
100%		200/200 [00:00<00:00,
952.87file/s]		
caesar_salad	:	
100%		200/200 [00:00<00:00,
939.24file/s]		
cannoli	:	
100%		200/200 [00:00<00:00,
923.83file/s]		
caprese_salad	:	
100%		200/200 [00:00<00:00,
904.33file/s]		
carrot_cake	:	
100%		200/200 [00:00<00:00,
878.78file/s]		
ceviche	:	
100%		200/200 [00:00<00:00,
946.50file/s]		
cheese_plate	:	
100%		200/200 [00:00<00:00,
1019.36file/s]		
cheesecake	:	
100%		200/200 [00:00<00:00,
1016.11file/s]		

```

chicken_curry      :
100%|              | 200/200 [00:00<00:00,
945.72file/s]
chicken_quesadilla :
100%|              | 200/200 [00:00<00:00,
955.36file/s]
chicken_wings      :
100%|              | 200/200 [00:00<00:00,
846.69file/s]
chocolate_cake     :
100%|              | 200/200 [00:00<00:00,
901.46file/s]
chocolate_mousse   :
100%|              | 200/200 [00:00<00:00,
964.25file/s]
churros            :
100%|              | 200/200 [00:00<00:00,
871.72file/s]
clam_chowder       :
100%|              | 200/200 [00:00<00:00,
1023.91file/s]
club_sandwich       :
100%|              | 200/200 [00:00<00:00,
924.95file/s]
crab_cakes         :
100%|              | 200/200 [00:00<00:00,
941.71file/s]
creme_brulee       :
100%|              | 200/200 [00:00<00:00,
982.50file/s]
croque_madame      :
100%|              | 200/200 [00:00<00:00,
966.81file/s]
cup_cakes          :
100%|              | 200/200 [00:00<00:00,
1020.92file/s]
deviled_eggs       :
100%|              | 200/200 [00:00<00:00,
970.53file/s]
donuts             :
100%|              | 200/200 [00:00<00:00,
954.84file/s]
dumplings          :
100%|              | 200/200 [00:00<00:00,
1000.46file/s]
edamame            :
100%|              | 200/200 [00:00<00:00,
985.40file/s]

```

```

eggs_benedict      :
100%|              | 200/200 [00:00<00:00,
1028.76file/s]
escargots          :
100%|              | 200/200 [00:00<00:00,
969.94file/s]
falafel            :
100%|              | 200/200 [00:00<00:00,
927.99file/s]
filet_mignon       :
100%|              | 200/200 [00:00<00:00,
967.99file/s]
fish_and_chips     :
100%|              | 200/200 [00:00<00:00,
856.59file/s]
foie_gras          :
100%|              | 200/200 [00:00<00:00,
1018.99file/s]
french_fries       :
100%|              | 200/200 [00:00<00:00,
986.89file/s]
french_onion_soup  :
100%|              | 200/200 [00:00<00:00,
935.47file/s]
french_toast       :
100%|              | 200/200 [00:00<00:00,
883.31file/s]
fried_calamari     :
100%|              | 200/200 [00:00<00:00,
953.20file/s]
fried_rice         :
100%|              | 200/200 [00:00<00:00,
944.01file/s]
frozen_yogurt      :
100%|              | 200/200 [00:00<00:00,
964.10file/s]
garlic_bread       :
100%|              | 200/200 [00:00<00:00,
855.71file/s]
gnocchi            :
100%|              | 200/200 [00:00<00:00,
982.59file/s]
greek_salad        :
100%|              | 200/200 [00:00<00:00,
905.75file/s]
grilled_cheese_sandwich:
100%|              | 200/200 [00:00<00:00,
961.98file/s]

```

```

grilled_salmon      :
100%|                | 200/200 [00:00<00:00,
943.86file/s]
guacamole           :
100%|                | 200/200 [00:00<00:00,
935.27file/s]
gyoza               :
100%|                | 200/200 [00:00<00:00,
918.79file/s]
hamburger           :
100%|                | 200/200 [00:00<00:00,
964.37file/s]
hot_and_sour_soup   :
100%|                | 200/200 [00:00<00:00,
793.54file/s]
hot_dog             :
100%|                | 200/200 [00:00<00:00,
733.84file/s]
huevos_rancheros    :
100%|                | 200/200 [00:00<00:00,
697.78file/s]
hummus              :
100%|                | 200/200 [00:00<00:00,
712.16file/s]
ice_cream           :
100%|                | 200/200 [00:00<00:00,
1012.03file/s]
lasagna             :
100%|                | 200/200 [00:00<00:00,
912.67file/s]
lobster_bisque      :
100%|                | 200/200 [00:00<00:00,
1019.14file/s]
lobster_roll_sandwich :
100%|                | 200/200 [00:00<00:00,
961.29file/s]
macaroni_and_cheese :
100%|                | 200/200 [00:00<00:00,
927.22file/s]
macarons            :
100%|                | 200/200 [00:00<00:00,
1020.16file/s]
miso_soup           :
100%|                | 200/200 [00:00<00:00,
1075.35file/s]
mussels             :
100%|                | 200/200 [00:00<00:00,
970.48file/s]

```


nachos	:	
100%		200/200 [00:00<00:00,
896.01file/s]		
omelette	:	
100%		200/200 [00:00<00:00,
933.60file/s]		
onion_rings	:	
100%		200/200 [00:00<00:00,
957.34file/s]		
oysters	:	
100%		200/200 [00:00<00:00,
918.55file/s]		
pad_thai	:	
100%		200/200 [00:00<00:00,
942.10file/s]		
paella	:	
100%		200/200 [00:00<00:00,
883.27file/s]		
pancakes	:	
100%		200/200 [00:00<00:00,
941.52file/s]		
panna_cotta	:	
100%		200/200 [00:00<00:00,
1034.50file/s]		
peking_duck	:	
100%		200/200 [00:00<00:00,
996.97file/s]		
pho	:	
100%		200/200 [00:00<00:00,
1000.61file/s]		
pizza	:	
100%		200/200 [00:00<00:00,
945.48file/s]		
pork_chop	:	
100%		200/200 [00:00<00:00,
987.56file/s]		
poutine	:	
100%		200/200 [00:00<00:00,
942.73file/s]		
prime_rib	:	
100%		200/200 [00:00<00:00,
828.66file/s]		
pulled_pork_sandwich	:	
100%		200/200 [00:00<00:00,
872.88file/s]		
ramen	:	
100%		200/200 [00:00<00:00,
920.10file/s]		

ravioli	:	
100%		200/200 [00:00<00:00,
945.41file/s]		
red_velvet_cake	:	
100%		200/200 [00:00<00:00,
949.75file/s]		
risotto	:	
100%		200/200 [00:00<00:00,
943.00file/s]		
samosa	:	
100%		200/200 [00:00<00:00,
970.33file/s]		
sashimi	:	
100%		200/200 [00:00<00:00,
983.94file/s]		
scallops	:	
100%		200/200 [00:00<00:00,
969.04file/s]		
seaweed_salad	:	
100%		200/200 [00:00<00:00,
881.75file/s]		
shrimp_and_grits	:	
100%		200/200 [00:00<00:00,
921.43file/s]		
spaghetti_bolognese	:	
100%		200/200 [00:00<00:00,
853.97file/s]		
spaghetti_carbonara	:	
100%		200/200 [00:00<00:00,
840.27file/s]		
spring_rolls	:	
100%		200/200 [00:00<00:00,
966.51file/s]		
steak	:	
100%		200/200 [00:00<00:00,
935.94file/s]		
strawberry_shortcake	:	
100%		200/200 [00:00<00:00,
978.83file/s]		
sushi	:	
100%		200/200 [00:00<00:00,
964.78file/s]		
tacos	:	
100%		200/200 [00:00<00:00,
922.80file/s]		
takoyaki	:	
100%		200/200 [00:00<00:00,
921.17file/s]		

```

tiramisu          :
100%|              | 200/200 [00:00<00:00,
940.52file/s]
tuna_tartare      :
100%|              | 200/200 [00:00<00:00,
940.16file/s]
waffles           :
100%|              | 200/200 [00:00<00:00,
930.80file/s]

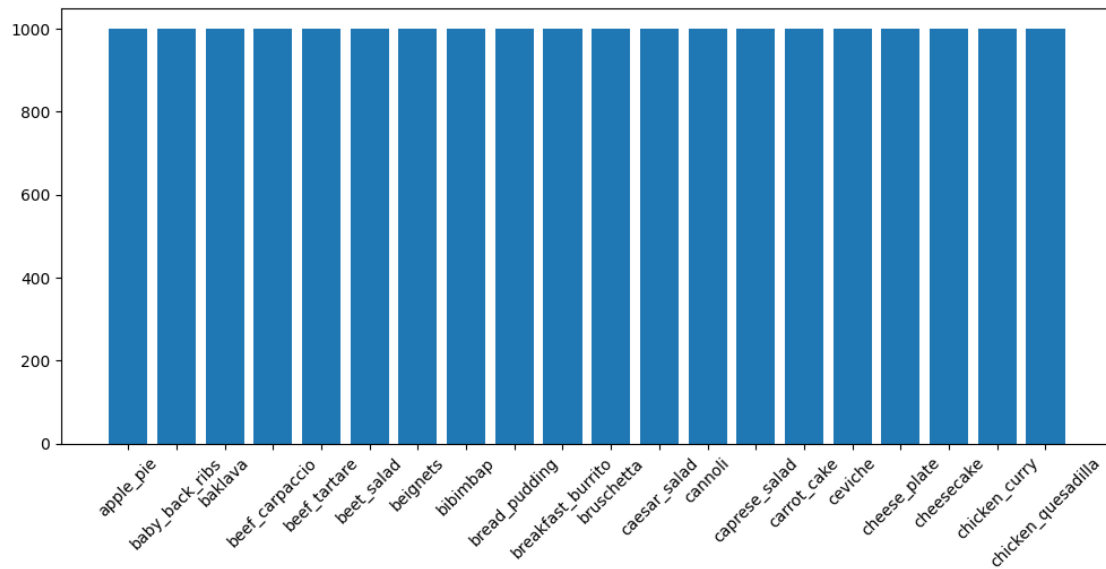
```

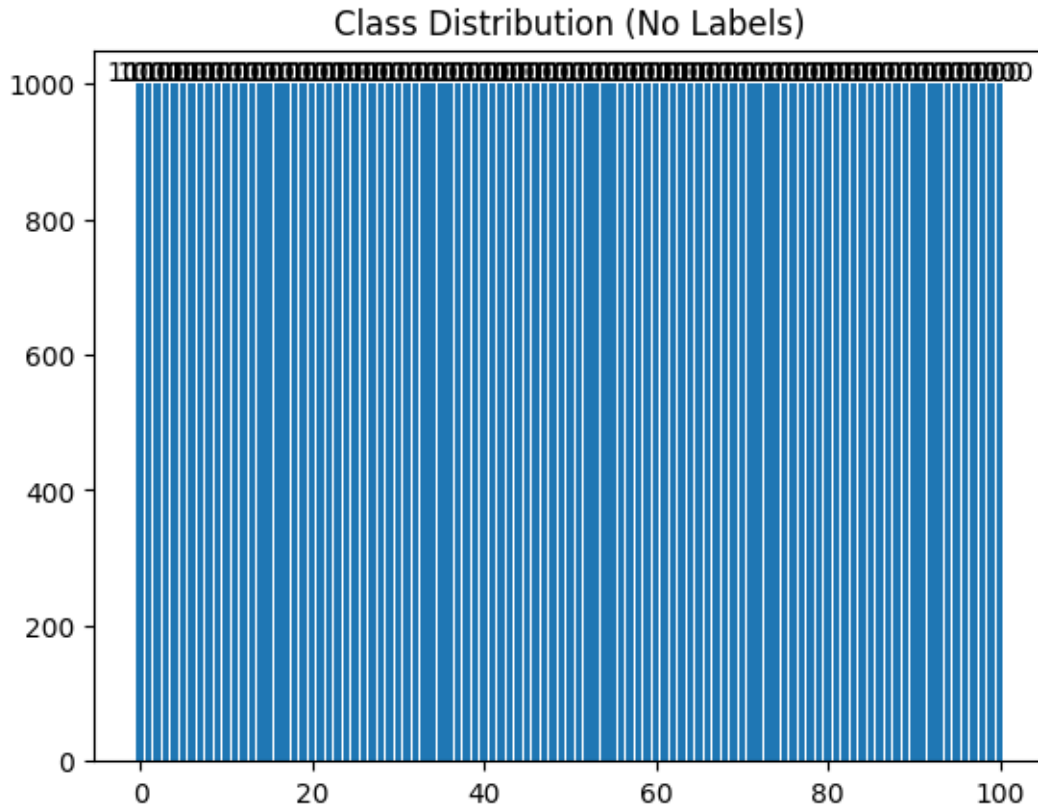
101 kind of images were found in the dataset

101000 images were found in the dataset.

16158 for training, 4040 for validation

average image height= 473 average image width= 493 aspect ratio h/w=
0.9594320486815415





```
[4]: class MyDataset(Dataset):
    def __init__(self, image_path, image_labels, transform=None):
        self.image_path = image_path
        self.image_class = image_labels
        self.transform = transform

    def __len__(self):
        return len(self.image_path)

    def __getitem__(self, item):
        image = Image.open(self.image_path[item])

        if image.mode != 'RGB':
            raise ValueError(f'image: {self.image_path[item]} is not a RGB_
↪image!')

        label = self.image_class[item]
        if self.transform is not None:
            image = self.transform(image)
```

```

        return image, label

    @staticmethod
    def collate_fn(batch):
        images, labels = tuple(zip(*batch))
        images = torch.stack(images, dim=0)
        labels = torch.as_tensor(labels)
        return images, labels

```

```

[5]: device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
device

```

```

[5]: device(type='cuda', index=0)

```

```

[6]: data_transform = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(200),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'valid': transforms.Compose([
        transforms.Resize(200),
        transforms.CenterCrop(200),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

```

```

[7]: batch_size = 16

# Menyiapkan Dataset dan DataLoader untuk Training
train_dataset = MyDataset(train_image_path, train_image_label,
    ↳transform=data_transform['train'])
trainloader = DataLoader(
    train_dataset,
    shuffle=True,
    batch_size=batch_size,
    num_workers=0,
    collate_fn=train_dataset.collate_fn
)

# Menyiapkan Dataset dan DataLoader untuk Validation
valid_dataset = MyDataset(val_image_path, val_image_label,
    ↳transform=data_transform['valid'])
validloader = DataLoader(
    valid_dataset,

```

```

        shuffle=False,
        batch_size=batch_size,
        num_workers=0,
        collate_fn=valid_dataset.collate_fn
    )

```

```

[8]: # ReLU -> BatchNorm -> Conv2d
class ConvBlock(nn.Module):
    def __init__(self, input_channels, num_channels):
        super().__init__()
        self.features = nn.Sequential(
            nn.ReLU(inplace=True),
            nn.BatchNorm2d(input_channels),
            nn.Conv2d(input_channels, num_channels, kernel_size=3, padding=1)
        )

    def forward(self, x):
        y = self.features(x)
        return y

class DenseBlock(nn.Module):
    def __init__(self, num_convs, input_channels, num_channels):
        super().__init__()
        layers = []
        for i in range(num_convs):
            layers.append(ConvBlock(num_channels * i + input_channels,
↳ num_channels))

        self.net = nn.Sequential(*layers)

    def forward(self, x):
        for blk in self.net:
            y = blk(x)
            # Menghubungkan input dengan output (concatenation) pada dimensi
↳ channel
            x = torch.concat([x, y], dim=1)
        return x

# ReLU -> BatchNorm -> Conv1x1 -> AvgPool
class transition_block(nn.Module):
    def __init__(self, input_channels, num_channels):
        super().__init__()
        self.features = nn.Sequential(
            nn.ReLU(inplace=True),
            nn.BatchNorm2d(input_channels),
            nn.Conv2d(input_channels, num_channels, kernel_size=1),
            nn.AvgPool2d(2, 2, ceil_mode=True)

```

```

    )

    def forward(self, x):
        y = self.features(x)
        return y

```

```

[9]: layer_1 = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3), nn.BatchNorm2d(64),
    nn.ReLU(inplace=True), nn.MaxPool2d(3, 2, padding=1, ceil_mode=True)
)

```

```

[10]: def DenseNet_121(num_classes):
    num_channels, growth_rate = 64, 32
    num_convs_in_dense_blocks = [6, 12, 24, 16]
    blks = []

    for i, num_convs in enumerate(num_convs_in_dense_blocks):
        blks.append(DenseBlock(num_convs, num_channels, growth_rate))
        num_channels += num_convs * growth_rate

        if i != len(num_convs_in_dense_blocks) - 1:
            blks.append(transition_block(num_channels, num_channels // 2))
            num_channels = num_channels // 2

    net = nn.Sequential(
        layer_1,
        *blks,
        nn.BatchNorm2d(num_channels),
        nn.ReLU(inplace=True),
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Flatten(),
        nn.Linear(num_channels, num_classes)
    )

    return net

# Contoh cara panggil:
# model = DenseNet_121(num_classes=2).to(device)

```

```

[11]: net = DenseNet_121(num_classes=len(class_indices)).to(device)
# summary(net, input_size=(batch_size, 3, 200, 200))

```

```

[12]: print(len(train_dataset), len(valid_dataset))

```

16158 4040

```

[13]: initial_lr = 0.001
epochs = 20
best_val_acc = 0.0
save_path = './DenseNet121.pth'

# Load model jika file sudah ada
if os.path.exists(save_path):
    □
    ↪ print('-----loading-----')
    net.load_state_dict(torch.load(save_path))

# Optimizer dan Loss Function
optimizer = torch.optim.Adam(net.parameters(), lr=initial_lr, weight_decay=0.
    ↪ 0001)
loss_function = nn.CrossEntropyLoss()

train_num = len(train_dataset)
valid_num = len(valid_dataset)

# List untuk menyimpan history
train_Acc, train_Loss, val_Acc, val_Loss = [], [], [], []

for epoch in range(epochs):
    # Training Phase
    running_loss = 0.0
    train_acc = 0.0
    val_acc = 0.0
    val_loss = 0.0

    net.train()
    train_bar = tqdm(trainloader, file=sys.stdout, colour='red')
    for step, data in enumerate(train_bar):
        images, labels = data
        optimizer.zero_grad()

        outputs = net(images.to(device))
        loss = loss_function(outputs, labels.to(device))
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        train_acc += (torch.argmax(outputs, dim=1) == labels.to(device)).sum().
        ↪ item()

        train_bar.desc = f'epoch [{epoch+1} / {epochs}], loss {loss.item():.3f}'

    train_accuracy = train_acc / train_num

```



```

train_loss = running_loss / train_num

# Validation Phase
net.eval()
with torch.no_grad():
    val_bar = tqdm(validloader, file=sys.stdout, colour='blue')
    for data in val_bar:
        images, labels = data
        outputs = net(images.to(device))
        loss = loss_function(outputs, labels.to(device))

        val_loss += loss.item()
        val_acc += (torch.argmax(outputs, dim=1) == labels.to(device)).
↪sum().item()

        val_bar.desc = f'epoch [{epoch+1} / {epochs}], loss {loss.item():.
↪3f}'

    valid_accuracy = val_acc / valid_num
    valid_loss = val_loss / valid_num

# Simpan History
train_Acc.append(train_accuracy)
train_Loss.append(train_loss)
val_Acc.append(valid_accuracy)
val_Loss.append(valid_loss)

print(f'epoch [{epoch+1} / {epochs}], train_loss: {train_loss:.3f},
↪train_accuracy: {train_accuracy:.3f}, '
      f'valid_loss: {valid_loss:.3f}, valid_accuracy: {valid_accuracy:.3f}')

# Save Best Model
if valid_accuracy > best_val_acc:
    best_val_acc = valid_accuracy
    torch.save(net.state_dict(), save_path)

print('Finished Training !')

```

```

epoch [1 / 20], loss 3.958: 100%|          | 1010/1010 [04:58<00:00,
3.38it/s]
epoch [1 / 20], loss 4.196: 100%|          | 253/253 [00:32<00:00,
7.73it/s]
epoch [1 / 20], train_loss: 0.289, train_accuracy: 0.030, valid_loss: 0.278,
valid_accuracy: 0.050
epoch [2 / 20], loss 3.640: 100%|          | 1010/1010 [04:55<00:00,
3.42it/s]
epoch [2 / 20], loss 4.116: 100%|          | 253/253 [00:33<00:00,

```

```

7.62it/s]
epoch [2 / 20], train_loss: 0.272, train_accuracy: 0.051, valid_loss: 63.918,
valid_accuracy: 0.067
epoch [3 / 20], loss 4.563: 100%|      | 1010/1010 [04:58<00:00,
3.38it/s]
epoch [3 / 20], loss 4.082: 100%|      | 253/253 [00:33<00:00,
7.65it/s]
epoch [3 / 20], train_loss: 0.263, train_accuracy: 0.069, valid_loss: 2912.229,
valid_accuracy: 0.074
epoch [4 / 20], loss 3.942: 100%|      | 1010/1010 [05:01<00:00,
3.35it/s]
epoch [4 / 20], loss 3.843: 100%|      | 253/253 [00:35<00:00,
7.19it/s]
epoch [4 / 20], train_loss: 0.255, train_accuracy: 0.085, valid_loss: 74403.918,
valid_accuracy: 0.101
epoch [5 / 20], loss 4.241: 100%|      | 1010/1010 [04:56<00:00,
3.41it/s]
epoch [5 / 20], loss 3.932: 100%|      | 253/253 [00:34<00:00,
7.32it/s]
epoch [5 / 20], train_loss: 0.249, train_accuracy: 0.100, valid_loss: 11.591,
valid_accuracy: 0.105
epoch [6 / 20], loss 3.979: 100%|      | 1010/1010 [04:57<00:00,
3.39it/s]
epoch [6 / 20], loss 3.416: 100%|      | 253/253 [00:33<00:00,
7.48it/s]
epoch [6 / 20], train_loss: 0.242, train_accuracy: 0.118, valid_loss: 0.251,
valid_accuracy: 0.119
epoch [7 / 20], loss 3.672: 100%|      | 1010/1010 [04:53<00:00,
3.44it/s]
epoch [7 / 20], loss 3.904: 100%|      | 253/253 [00:33<00:00,
7.57it/s]
epoch [7 / 20], train_loss: 0.236, train_accuracy: 0.135, valid_loss: 0.413,
valid_accuracy: 0.158
epoch [8 / 20], loss 3.936: 100%|      | 1010/1010 [04:52<00:00,
3.45it/s]
epoch [8 / 20], loss 3.297: 100%|      | 253/253 [00:32<00:00,
7.84it/s]
epoch [8 / 20], train_loss: 0.229, train_accuracy: 0.155, valid_loss: 0.232,
valid_accuracy: 0.144
epoch [9 / 20], loss 3.688: 100%|      | 1010/1010 [06:11<00:00,
2.72it/s]
epoch [9 / 20], loss 3.423: 100%|      | 253/253 [00:32<00:00,
7.79it/s]
epoch [9 / 20], train_loss: 0.223, train_accuracy: 0.169, valid_loss: 0.232,
valid_accuracy: 0.203
epoch [10 / 20], loss 3.761: 100%|     | 1010/1010 [04:52<00:00,
3.46it/s]
epoch [10 / 20], loss 3.347: 100%|     | 253/253 [00:32<00:00,

```

```

7.78it/s]
epoch [10 / 20], train_loss: 0.217, train_accuracy: 0.188, valid_loss: 0.529,
valid_accuracy: 0.210
epoch [11 / 20], loss 2.936: 100%|      | 1010/1010 [04:51<00:00,
3.46it/s]
epoch [11 / 20], loss 3.344: 100%|      | 253/253 [00:32<00:00,
7.90it/s]
epoch [11 / 20], train_loss: 0.211, train_accuracy: 0.205, valid_loss: 0.208,
valid_accuracy: 0.237
epoch [12 / 20], loss 2.918: 100%|      | 1010/1010 [04:51<00:00,
3.46it/s]
epoch [12 / 20], loss 2.829: 100%|      | 253/253 [00:31<00:00,
8.06it/s]
epoch [12 / 20], train_loss: 0.207, train_accuracy: 0.220, valid_loss: 0.238,
valid_accuracy: 0.263
epoch [13 / 20], loss 3.532: 100%|      | 1010/1010 [04:47<00:00,
3.51it/s]
epoch [13 / 20], loss 3.353: 100%|      | 253/253 [00:30<00:00,
8.22it/s]
epoch [13 / 20], train_loss: 0.202, train_accuracy: 0.236, valid_loss: 111.262,
valid_accuracy: 0.243
epoch [14 / 20], loss 2.318: 100%|      | 1010/1010 [04:51<00:00,
3.47it/s]
epoch [14 / 20], loss 2.998: 100%|      | 253/253 [00:31<00:00,
7.95it/s]
epoch [14 / 20], train_loss: 0.198, train_accuracy: 0.245, valid_loss: 1.239,
valid_accuracy: 0.251
epoch [15 / 20], loss 3.033: 100%|      | 1010/1010 [04:50<00:00,
3.48it/s]
epoch [15 / 20], loss 3.134: 100%|      | 253/253 [00:32<00:00,
7.85it/s]
epoch [15 / 20], train_loss: 0.193, train_accuracy: 0.265, valid_loss: 0.525,
valid_accuracy: 0.289
epoch [16 / 20], loss 2.718: 100%|      | 1010/1010 [04:50<00:00,
3.47it/s]
epoch [16 / 20], loss 3.216: 100%|      | 253/253 [00:32<00:00,
7.77it/s]
epoch [16 / 20], train_loss: 0.188, train_accuracy: 0.280, valid_loss: 0.200,
valid_accuracy: 0.263
epoch [17 / 20], loss 2.782: 100%|      | 1010/1010 [04:49<00:00,
3.49it/s]
epoch [17 / 20], loss 2.778: 100%|      | 253/253 [00:31<00:00,
8.08it/s]
epoch [17 / 20], train_loss: 0.183, train_accuracy: 0.293, valid_loss: 0.189,
valid_accuracy: 0.296
epoch [18 / 20], loss 3.251: 100%|      | 1010/1010 [04:47<00:00,
3.52it/s]
epoch [18 / 20], loss 3.633: 100%|      | 253/253 [00:30<00:00,

```

```

8.33it/s]
epoch [18 / 20], train_loss: 0.181, train_accuracy: 0.297, valid_loss: 0.201,
valid_accuracy: 0.284
epoch [19 / 20], loss 2.684: 100%|      | 1010/1010 [04:45<00:00,
3.53it/s]
epoch [19 / 20], loss 2.705: 100%|      | 253/253 [00:30<00:00,
8.32it/s]
epoch [19 / 20], train_loss: 0.177, train_accuracy: 0.318, valid_loss: 0.184,
valid_accuracy: 0.314
epoch [20 / 20], loss 3.106: 100%|      | 1010/1010 [04:45<00:00,
3.54it/s]
epoch [20 / 20], loss 2.797: 100%|      | 253/253 [00:30<00:00,
8.18it/s]
epoch [20 / 20], train_loss: 0.174, train_accuracy: 0.326, valid_loss: 0.180,
valid_accuracy: 0.336
Finished Training !

```

```

[16]: def plot_auc_acc_loss(epochs):
    Epochs = [i for i in range(epochs)]

    plt.style.use('fivethirtyeight')
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(16, 8))

    # Plot Training & Validation Loss
    axes[0].plot(Epochs, train_Loss, 'r', label='Training loss')
    axes[0].plot(Epochs, val_Loss, 'g', label='Validation loss')
    axes[0].set_title('Training and Validation Loss')
    axes[0].set_xlabel('Epochs', fontsize=18)
    axes[0].set_ylabel('Loss', fontsize=18)
    axes[0].legend()

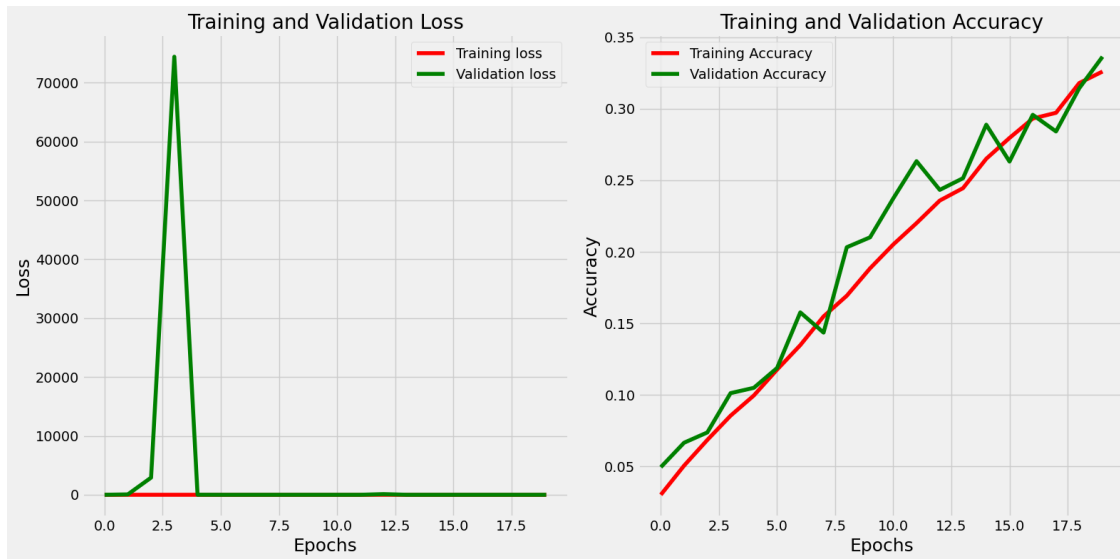
    # Plot Training & Validation Accuracy
    axes[1].plot(Epochs, train_Acc, 'r', label='Training Accuracy')
    axes[1].plot(Epochs, val_Acc, 'g', label='Validation Accuracy')
    axes[1].set_title('Training and Validation Accuracy')
    axes[1].set_xlabel('Epochs', fontsize=18)
    axes[1].set_ylabel('Accuracy', fontsize=18)
    axes[1].legend()

    plt.tight_layout()
    plt.show()

    return Epochs

# Menjalankan fungsi plot
Epochs = plot_auc_acc_loss(epochs=20)

```



```
[15]: try:
        with open('./classes_indices.json', 'r') as json_file:
            class_indict = json.load(json_file)
    except Exception as e:
        print(e)
        exit(-1)

    def predictor(testloader):
        errors = 0
        y_pred, y_true = [], []

        # Inisialisasi model dan load weights
        net = DenseNet_121(num_classes=len(class_indict)).to(device)
        net_weight_path = './DenseNet121.pth'
        net.load_state_dict(torch.load(net_weight_path, map_location=device))

        net.eval()
        with torch.no_grad():
            for data in testloader:
                images, labels = data
                images = images.to(device)
                labels = labels.to(device)

                outputs = net(images)
                preds = torch.argmax(torch.softmax(outputs, dim=1), dim=1)

                for i in range(len(preds)):
                    y_pred.append(preds[i].cpu().item())
```

```

        y_true.append(labels[i].cpu().item())

tests = len(y_pred)

for i in range(tests):
    if y_pred[i] != y_true[i]:
        errors += 1

acc = (1 - errors / tests) * 100
print(f'there were {errors} errors in {tests} tests for an accuracy of {acc:
↪.2f}%')

ypred = np.array(y_pred)
ytrue = np.array(y_true)

f1score = f1_score(ytrue, ypred, average='weighted') * 100

class_count = len(list(class_indict.values()))
classes = list(class_indict.values())

cm = confusion_matrix(ytrue, ypred)

# Visualisasi Confusion Matrix
plt.figure(figsize=(16, 8))

plt.subplot(1, 2, 1)
sns.heatmap(cm, annot=False, cmap='Blues')
plt.title("Confusion Matrix")

plt.subplot(1, 2, 2)
sns.heatmap(cm / np.sum(cm), annot=False, fmt='.1%')
plt.title("Normalized Confusion Matrix")

plt.show()

# Classification Report
clr = classification_report(ytrue, ypred, target_names=classes, digits=4)
print("Classification Report:\n", clr)

return f1score

# Menjalankan fungsi predictor
f1score = predictor(validloader)

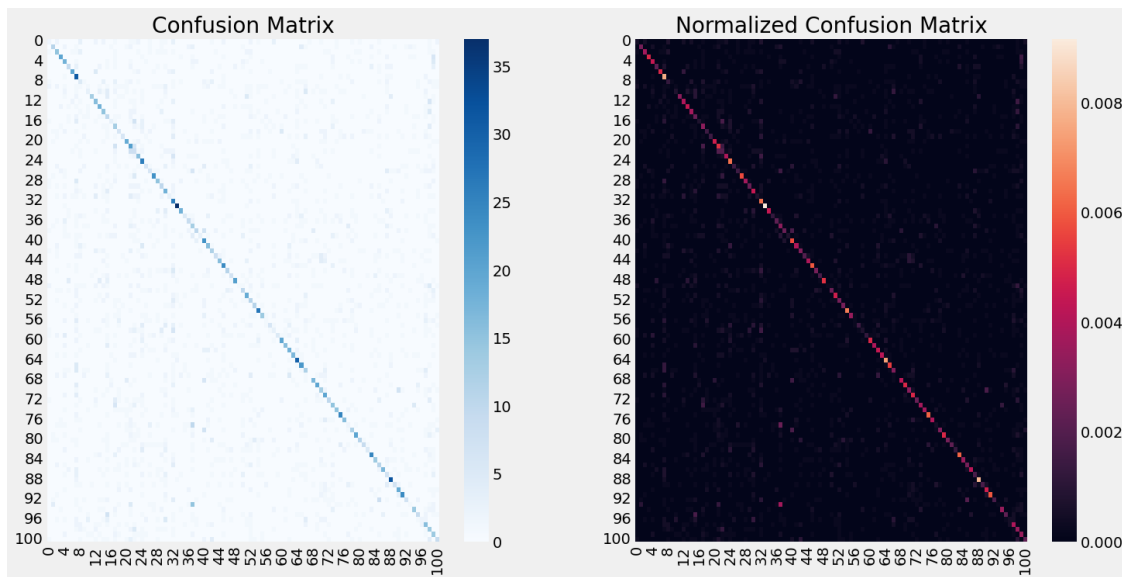
```

/tmp/ipykernel_21361/3185670908.py:15: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See

<https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
net.load_state_dict(torch.load(net_weight_path, map_location=device))
```

there were 2681 errors in 4040 tests for an accuracy of 33.64%



Classification Report:

	precision	recall	f1-score	support
apple_pie	0.1176	0.0500	0.0702	40
baby_back_ribs	0.5000	0.2750	0.3548	40
baklava	0.3409	0.3750	0.3571	40
beef_carpaccio	0.6897	0.5000	0.5797	40
beef_tartare	0.2769	0.4500	0.3429	40
beet_salad	0.3750	0.2250	0.2812	40
beignets	0.7407	0.5000	0.5970	40
bibimbap	0.2385	0.7750	0.3647	40
bread_pudding	0.1163	0.1250	0.1205	40
breakfast_burrito	0.0000	0.0000	0.0000	40
bruschetta	0.1579	0.0750	0.1017	40
caesar_salad	0.8125	0.3250	0.4643	40
cannoli	0.3265	0.4000	0.3596	40

caprese_salad	0.4286	0.4500	0.4390	40
carrot_cake	0.3191	0.3750	0.3448	40
ceviche	0.1176	0.2500	0.1600	40
cheese_plate	0.2727	0.0750	0.1176	40
cheesecake	0.1733	0.3250	0.2261	40
chicken_curry	0.1818	0.1500	0.1644	40
chicken_quesadilla	0.5000	0.1500	0.2308	40
chicken_wings	0.4524	0.4750	0.4634	40
chocolate_cake	0.2211	0.5250	0.3111	40
chocolate_mousse	0.1039	0.2000	0.1368	40
churros	0.3478	0.4000	0.3721	40
clam_chowder	0.2955	0.6500	0.4062	40
club_sandwich	0.3000	0.1500	0.2000	40
crab_cakes	0.1875	0.1500	0.1667	40
creme_brulee	0.6389	0.5750	0.6053	40
croque_madame	0.3947	0.3750	0.3846	40
cup_cakes	0.2564	0.2500	0.2532	40
deviled_eggs	0.2143	0.4500	0.2903	40
donuts	0.2727	0.0750	0.1176	40
dumplings	0.2047	0.6500	0.3114	40
edamame	0.8222	0.9250	0.8706	40
eggs_benedict	0.6000	0.4500	0.5143	40
escargots	0.3571	0.1250	0.1852	40
falafel	0.3600	0.2250	0.2769	40
filet_mignon	0.1688	0.3250	0.2222	40
fish_and_chips	0.3846	0.2500	0.3030	40
foie_gras	0.6000	0.0750	0.1333	40
french_fries	0.3333	0.5750	0.4220	40
french_onion_soup	0.3721	0.4000	0.3855	40
french_toast	0.2667	0.3000	0.2824	40
fried_calamari	0.4706	0.2000	0.2807	40
fried_rice	0.4571	0.4000	0.4267	40
frozen_yogurt	0.3478	0.6000	0.4404	40
garlic_bread	0.5238	0.2750	0.3607	40
gnocchi	0.2593	0.1750	0.2090	40
greek_salad	0.6562	0.5250	0.5833	40
grilled_cheese_sandwich	0.5000	0.0250	0.0476	40
grilled_salmon	0.2439	0.2500	0.2469	40
guacamole	0.4043	0.4750	0.4368	40
gyoza	0.1507	0.2750	0.1947	40
hamburger	0.4783	0.2750	0.3492	40
hot_and_sour_soup	0.4500	0.6750	0.5400	40
hot_dog	0.4667	0.3500	0.4000	40
huevos_rancheros	0.0909	0.0250	0.0392	40
hummus	0.4286	0.1500	0.2222	40
ice_cream	0.0909	0.0750	0.0822	40
lasagna	0.5556	0.1250	0.2041	40
lobster_bisque	0.6452	0.5000	0.5634	40

lobster_roll_sandwich	0.5714	0.4000	0.4706	40
macaroni_and_cheese	0.3273	0.4500	0.3789	40
macarons	0.5667	0.4250	0.4857	40
miso_soup	0.3704	0.7500	0.4959	40
mussels	0.4583	0.5500	0.5000	40
nachos	0.2069	0.3000	0.2449	40
omelette	0.0000	0.0000	0.0000	40
onion_rings	0.5000	0.4250	0.4595	40
oysters	0.4878	0.5000	0.4938	40
pad_thai	0.2766	0.3250	0.2989	40
paella	0.3030	0.5000	0.3774	40
pancakes	0.4762	0.2500	0.3279	40
panna_cotta	0.2885	0.3750	0.3261	40
peking_duck	0.5357	0.3750	0.4412	40
pho	0.5319	0.6250	0.5747	40
pizza	0.5517	0.4000	0.4638	40
pork_chop	0.0870	0.0500	0.0635	40
poutine	0.2857	0.3000	0.2927	40
prime_rib	0.5882	0.5000	0.5405	40
pulled_pork_sandwich	0.5000	0.2750	0.3548	40
ramen	0.5294	0.2250	0.3158	40
ravioli	0.5556	0.1250	0.2041	40
red_velvet_cake	0.5000	0.6250	0.5556	40
risotto	0.5000	0.3500	0.4118	40
samosa	0.2157	0.2750	0.2418	40
sashimi	0.7391	0.4250	0.5397	40
scallops	0.1346	0.1750	0.1522	40
seaweed_salad	0.5246	0.8000	0.6337	40
shrimp_and_grits	0.4375	0.1750	0.2500	40
spaghetti_bolognese	0.4043	0.4750	0.4368	40
spaghetti_carbonara	0.5333	0.6000	0.5647	40
spring_rolls	0.3333	0.1750	0.2295	40
steak	0.1111	0.0250	0.0408	40
strawberry_shortcake	0.4667	0.3500	0.4000	40
sushi	0.2903	0.2250	0.2535	40
tacos	0.2222	0.0500	0.0816	40
takoyaki	0.2319	0.4000	0.2936	40
tiramisu	0.1707	0.3500	0.2295	40
tuna_tartare	0.1795	0.3500	0.2373	40
waffles	0.3600	0.2250	0.2769	40
accuracy			0.3364	4040
macro_avg	0.3704	0.3364	0.3253	4040
weighted_avg	0.3704	0.3364	0.3253	4040